

Endo_edge — Unified GI Endoscopy Report Schema and Fine-Tuning Setup

Qwen3-VL-8B-Instruct · GastroVision + HyperKvasir + LIMUC · 2026-08-30

Executive summary

This document records the target schema for the Endo_edge instruction fine-tune, the evidence behind every schema decision, the state of the training pipeline as it is currently running, and answers to the questions raised while building it. All figures were recomputed from the corpus rather than quoted from memory.

Item	Value
Corpus	29,938 images — GastroVision 8,000, HyperKvasir 10,662, LIMUC 11,276
Schema	9 keys, always all present; null = the source establishes no value
Distinct targets	43 (from 54 source classes; 0 within-dataset collisions = lossless)
Source of truth	reports/schema_key_presence.csv (29,938 rows x 25 columns)
Model	Qwen3-VL-8B-Instruct, vision tower frozen
Runs	full fine-tune -> LoRA r=32 -> QLoRA int4, chained, each from the BASE model
Hardware	1x GH200 on the login node, effective batch 44 via gradient accumulation

1. The target schema (v2)

Every image maps to one JSON object with nine keys. All nine are always present. A **null** means the source annotation does not establish a value — it is not a negative claim.

```
{
  "abnormality_present": true,
  "abnormality_type": "ulcerative_colitis",
  "grade": "2",
  "instrument_present": false,
  "landmark": null,
  "landmark_certain": false,
  "prep_quality": null,
  "procedure": null,
  "tract": "lower"
}
```

Value spaces

Key	Type	Values	Non-null
abnormality_present	bool null	—	26,747
abnormality_type	str null	12 findings: ulcerative_colitis, polyp, esophagitis, barretts, blood_in_lumen, colorectal_cancer, mucosal_inflammation, angiectasia, erythema, esophageal_varices, ulcer, hemorrhoids	9,284
grade	str null	esophagitis a b-d · barretts short-segment long-segment · UC 0-1 1 1-2 2 2-3 3	6,791
instrument_present	bool (never null)	—	1,266 true
landmark	str null	12: esophagus, gej_zline, stomach, retroflex_stomach, pylorus, duodenal_bulb, small_bowel_terminal_ileum, cecum, ileocecal_valve, large_bowel_mucosa, retroflex_rectum, colon_diverticula	8,863
landmark_certain	bool (never null)	false = anatomy UNKNOWN, not absent	—
prep_quality	str null	adequate, inadequate, impacted_stool	1,925
procedure	str null	dyed_lifted_polyp, dyed_resection_margin, resected_polyp, resection_margin	2,495
tract	str null	upper, lower	28,463

Cross-field invariants

JSON Schema cannot express these; they live in validate():

1. landmark != null <=> landmark_certain == true (both directions)
2. landmark implies its tract (retroflex_stomach -> upper)
3. abnormality_type != null => abnormality_present != false
4. abnormality_present == true => abnormality_type != null OR procedure != null
5. grade != null => type is set, is in GRADES, value in that type's set
6. prep_quality != null => tract == 'lower'

Invariant 4 is the one relaxation from v1: it lets a procedure row assert that something is present without naming a pathology.

Why landmark_certain exists

The three corpora are single-label: an image names either a landmark or a finding, never both. For a finding image the anatomy is therefore **unknown**, not absent. Encoding it as landmark=null with landmark_certain=false keeps the two cases apart. Training 'unknown' as 'no landmark' would teach the model that a finding implies no anatomy.

2. What changed from v1 (6 keys) to v2 (9 keys)

Nothing was removed. Two values moved to their own keys and three distinctions were recovered that v1 discarded. Verified lossless: every image's original source class is still recoverable from its target.

Change	Effect	Images
accessory_tools moved out of abnormality_type to instrument_present	neutral — a categorical value becomes a bool, same information	1,266
therapeutic moved to procedure AND split 1 -> 4	GAIN — 4 distinct procedures instead of one token	2,495
BBPS added as prep_quality	GAIN — previously discarded entirely	1,925
retroflex-stomach added as 12th landmark	GAIN — was merged into stomach	1,733
HyperKvasir barretts given grade long-segment	GAIN — HK distinguishes it from short-segment	136

Result: 36 -> 43 distinguishable targets from the same pixels and the same folder names. abnormality_type shrinks 14 -> 12 (accessory_tools and therapeutic leave); landmark grows 11 -> 12.

Why v1 needed fixing

Three concrete defects, all measured:

Defect	Detail
BBPS thrown away	1,925 images (6.4%) collapsed to one near-empty target asserting only tract='lower'. bbps-0-1 vs bbps-2-3 is adequate vs inadequate bowel prep — clinically decisive, and a HyperKvasir benchmark task in its own right. Worse than losing the label: it trained the model to emit an empty report for 1,925 varied colonoscopy images.
therapeutic merged procedures	4 2,495 images. Dyed-lifted-polyp (submucosal lift, lesion still present) and dyed-resection-margin (post-resection crater) are visually and clinically distinct.
abnormality_type non-abnormalities	held 3,761 images (12.6%). An instrument is not a pathology; a resection margin is a procedure state. This is also why abnormality_present was null on 3,191 rows — the schema could not say 'a tool is here, pathology unknown'.

3. Questions answered from the data

Q: How many images have all keys populated?

Zero — and it is structurally impossible. Verified four ways, including going back to the raw datasets rather than trusting the derived JSONL.

Non-null keys (of 9)	Images
9	0
6	6,779
5	13,654
4	8,239
2	1,266

Two of the nine keys are booleans that are never null, so of the 7 nullable keys the maximum ever populated is 4. The reason is that every image falls into exactly one content axis, never two:

landmark	abn_type	procedure	prep_quality	instrument	Images
	YES				9,284
YES					8,863
					6,105 (Mayo 0, normal)
		YES			2,495
			YES		1,925
				YES	1,266
any together	two				0

That is a strict partition of all 29,938 images. The corpus is not one multi-labelled dataset — it is six disjoint single-label sub-corpora stacked into one file. Adding keys therefore makes rows *more* sparse, never less. No schema over this data can produce a fully-populated sample.

Q: Do any of the datasets provide a landmark for finding images?

No — none of them. Checked every metadata file at full depth.

Dataset	Evidence
---------	----------

HyperKvasir	image-labels.csv is one row per image; Classification is a partition (anatomical-landmarks XOR pathological-findings XOR therapeutic XOR quality). The only other column, Organ, takes two values: Lower GI 7,210 / Upper GI 3,452 — that is the tract key, already extracted.
GastroVision	The QA CSV does contain 'What anatomical landmark is shown?' but it is asked ONLY of images already in a landmark folder (Cecum, Duodenal bulb, Ileocecal valve, Pylorus). Never of a finding image. The anatomy questions asked of findings are mechanically derived and demonstrably buggy: 65 images from the Gastric polyps folder answer 'Yes' to 'Are there any polyps in the colon?'
LIMUC	No metadata file anywhere in the tree. Folder name is the Mayo score.

Related finding: GastroVision finding class names do encode organ (Colon polyps 820, Colorectal cancer 139, Barrett's 95, Gastric polyps 65, Mucosal inflammation large bowel 29, Esophageal varices 7 = 1,155 images). That is genuine annotator intent, but it is organ-level and already captured by tract.

Q: Could landmark be filled in by clinical rule?

No — this was considered and rejected. It would be incorrect for 6,994 of the 7,000 candidate rows. Four of the eleven v1 landmark tokens have normality baked into the source class name:

```
"normal esophagus"          -> esophagus
"normal stomach"           -> stomach
"gastroesophageal_junction_normal z-line" -> gej_zline
"normal mucosa and vascular pattern in the large bowel" -> large_bowel_mucosa
```

Proposed rule	Rows	Why it fails
ulcerative_colitis large_bowel_mucosa	-> 6,022	The class is literally 'normal mucosa AND VASCULAR PATTERN'. Loss of vascular pattern defines Mayo >= 1. All 5,171 LIMUC graded rows are Mayo 1-3 — zero exceptions.
esophagitis/varices -> esophagus	777	Class is 'Normal esophagus'; esophagitis is inflamed esophagus.
barretts -> gej_zline	189	GastroVision's class is 'normal z-line'; Barrett's is the displaced, abnormal one.
hemorrhoids -> retroflex_rectum	6	VALID — 'retroflex rectum' is a view, not a normality claim.

A second, independent objection: the schema invariant forces landmark_certain=true whenever landmark is named. A rule-filled row would assert the image establishes anatomy that came from the label, not the pixels — the same shortcut-learning failure as the grade-provenance problem.

Q: Are all three datasets standardised now?

Yes. All 29,938 images across the three corpora emit the identical nine-key object; every row has all nine keys present. What varies is how many carry a value. 54 source classes map to 43 distinct targets with **zero within-dataset collisions** — the 11 remaining merges are two corpora naming the same clinical entity (e.g. LIMUC Mayo 1 + HyperKvasir ulcerative-colitis-grade-1), which is the point of a unified schema, not a loss.

Q: Why is a row like Mayo 0 null almost everywhere?

```
{"abnormality_present": false, "abnormality_type": null, "grade": null,
 "instrument_present": false, "landmark": null, "landmark_certain": false,
 "prep_quality": null, "procedure": null, "tract": "lower"}
```

That is LIMUC Mayo 0 — all 6,105 of them, exactly. LIMUC's only annotation is a Mayo score on a colonoscopy image; Mayo 0 means normal mucosa, no disease activity. The dataset establishes precisely two facts and the target says exactly those two. The false/null distinction is load-bearing:

Field	Value	Meaning
abnormality_present	false	positive claim — the source says no abnormality
landmark_certain	false	anatomy is UNKNOWN, not absent
instrument_present	false	no instrument visible
landmark, grade, ...	null	source establishes nothing

Q: Will bf16 weights run on a GTX 1080?

No — three separate blockers. A GTX 1080 is Pascal (GP104, compute capability 6.1, 8 GB).

Blocker	Detail
No bf16 hardware	bf16 needs Ampere (sm_80)+. Convert to fp32 instead — bf16 to fp32 is exact. Do NOT convert to fp16: bf16 has a wider exponent range so outlier weights can overflow to inf, and fp16 on GP104 runs at 1/64 rate anyway.
8 GB cannot hold 8B	fp32 ~32 GB, fp16/bf16 ~16 GB, int8 ~8 GB, int4 ~5 GB — before activations, KV cache, or the vision encoder. Only 4-bit gets close.
CUDA 13 dropped Pascal	The training env is torch 2.13.0+cu130. CUDA 13.0 removed Maxwell/Pascal/Volta support, so that build will not initialise on a 1080. bitsandbytes NF4 kernels also need sm_75+, so the QLoRA path does not rescue it either.

What works: llama.cpp / GGUF (supports Pascal, has Qwen3-VL mmproj vision support) quantized to Q4_K_M at roughly 5 GB, leaving ~3 GB for context. Otherwise CPU inference in fp32, or any Ampere+ card. If a 1080 is the real deployment target, the QLoRA run is the most relevant of the three since it is already trained under 4-bit quantization.

4. How training is set up now

The dataloader

reports/schema_key_presence.csv is the single source of truth. Each record carries the nine-key target rebuilt from its val_* columns, plus the list of keys the annotation establishes (has_* == yes), so the CSV and the training targets cannot drift. Note the jsonl/ files on disk are **stale** — they still hold the old 6-key targets and are not read by training.

```
batch keys: attention_mask, image_grid_thw, input_ids, labels,
             mm_token_type_ids, pixel_values
input_ids (8, 656)  labels (8, 656)
```

Loss is taken over the assistant JSON only; the system turn, image tokens, instruction and the generation header are masked to -100. Verified: decoding the unmasked span returns the target byte-for-byte.

Paired and unpaired conditioning

Every batch mixes two conditions. The role prompt is drawn once per batch from 25 variants; the paired flag is drawn per sample as a Bernoulli(0.5). Both draws are a deterministic function of (seed, global_step), so a resumed run reproduces the schedule it would have had.

Condition	System turn
unpaired	One of 25 role prompts, nothing more. The model must decide for itself which fields the image supports. This is what inference looks like.
paired	The same role prompt, extended with an explicit request for exactly the keys this sample establishes, and an instruction to leave the rest null.

Example of the paired extension for a landmark row:

```
You are a gastrointestinal endoscopy expert. Analyse the endoscopic
image and report your findings as JSON. For this image, determine
whether a pathological abnormality is present (`abnormality_present`),
whether an accessory instrument is visible (`instrument_present`), the
anatomical landmark shown (`landmark`), whether the anatomy is
established by the image (`landmark_certain`), and which GI tract the
image is from (`tract`). Leave every other field null.
```

The three runs

All three start from the **base** Qwen3-VL-8B-Instruct, not from each other — they are an ablation, not a curriculum. Vision tower frozen in all three (351 tensors).

Run	Mode	Precision	LR	Optimizer	Micro x accum
-----	------	-----------	----	-----------	---------------

full_fp16	full fine-tune	bf16	1e-5	AdamW 8-bit	4 x 11
lora_r32	LoRA r=32 a=64	bf16	1e-4	AdamW	11 x 4
qlora_int4	QLoRA nf4 + LoRA r=32	bf16 compute	2e-4	AdamW 8-bit	11 x 4

Batch composition and iteration count

One **batch** = one optimizer step = 44 samples. That 44 is deliberate: it matches the original 22/GPU x 2 GPUs, so moving to a single GPU changed nothing the optimizer sees.

```
full FT:      1 step = 44 samples = 11 forward/backward passes of 4
LoRA/QLoRA:  1 step = 44 samples = 4 forward/backward passes of 11
```

```
1 epoch  = 473 steps = 5,203 micro-iterations (full FT)
3 epochs = 1,419 steps = 15,609 micro-iterations
```

The split exists because the peak allocation is the logits tensor, batch x seq_len x 151,936 vocab. At 44 samples that is ~20 GB in one go, which OOM'd; at micro-batch 4 it is ~1.8 GB. The gradient maths is identical — each micro-loss is divided by accum before backward, so summing them gives exactly the mean over all 44 samples.

LoRA rank

r=32, alpha=64 on all seven linear projections (q,k,v,o,gate,up,down), vision tower frozen — 87,293,952 trainable parameters, 0.99% of 8.85B. The output space is a short constrained JSON over a closed vocabulary, so the task needs far less capacity than open-ended instruction tuning; r=8-16 would probably suffice and r=32 leaves headroom.

Hardware and precision

Setting	Value	Note
GPUs	1 (auto-selected as freest)	harness runs the script directly, no torchrun agent
Effective batch	44	identical to the previous 22 x 2
Epochs	3	reduced from 5
Steps per run	1,419	473 per epoch
Run 1 precision	bf16	FORCED — see below
Checkpoints	every 250 steps, keep last 3	+ adapter export for LoRA/QLoRA

Run 1 is bf16, not the fp16 mixed precision originally specified. This is the most consequential deviation and it was forced by the single-GPU decision. fp16 mixed precision requires fp32 master weights — that is what GradScaler unscales into. On one GPU with no FSDP sharding that is roughly 32 GB masters + 32 GB grads + 16 GB Adam = 80 GB static before activations, against 28-75 GB free on a shared GPU. Measured consequence: the model took 2h06m to load instead of seconds, because 32 GB of fp32 has to be materialised in host RAM. bf16 needs no master copy, runs at ~57 GB, and is the standard choice on Hopper — fp16 with loss scaling is the older workaround for hardware without bf16.

The harness

scripts/harness.py runs the three in order, one at a time. State lives in runs/harness_state.json, written atomically so a kill mid-write cannot corrupt it. If a run dies for any reason the harness relauches it; train_v2.py finds its own last checkpoint and resumes, so attempts climbs but step never goes backwards. A run is finished only when it writes DONE.json. Three launches with zero progress aborts rather than spinning.

This was exercised on a real interruption, not simulated:

```
[harness] full_fp16: exited rc=-15 at step 500 (was 0); relaunching, attempt 2
[harness] launching full_fp16 on GPU 1
  steps/epoch 473 total 1419
  RESUMED from runs/full_fp16/checkpoint-0000500 at step 500, epoch 1
```

Logging

Each run logs to its own wandb URL, with the run ID pinned and resume='allow' so an interruption continues the same run rather than forking a new one.

Run	URL
full_fp16	https://wandb.ai/ankitbelbase034-pulchowk-campus/Endoedge/runs/endoedge-full-fp16
lora_r32	https://wandb.ai/ankitbelbase034-pulchowk-campus/Endoedge/runs/endoedge-lora-r32
qlora_int4	https://wandb.ai/ankitbelbase034-pulchowk-campus/Endoedge/runs/endoedge-qlora-int4

Every step: loss, grad_norm, loss_ema (decay 0.98), loss_var (100-step window), lr, epoch, paired_frac, samples/s, accum_steps. Every 250 steps: validation loss, plus an 8-image inference table with image, paired flag, prediction, target and exact-match.

Caveat: train/paired_frac is computed from the last micro-batch only, so on the full-FT run it is a fraction out of 4 samples rather than 44 and will look quantised (0, 0.25, 0.5, 0.75, 1.0). The underlying Bernoulli draw is per sample across all 44.

5. Bugs found and fixed during bring-up

None of these were visible from reading the code; each was found by running it.

#	Bug	Fix
1	Triton JIT could not compile its CUDA shim: Python.h absent. Both environments are venvs whose base has no C headers, and the only system headers are Python 3.6. Would have blocked ALL THREE runs — torch 2.13's forward path uses a Triton-backed native op.	Symlinked the uv-managed CPython 3.12.13 headers (same version) into the base prefix include dir. Verified with a real Triton kernel: max err 0.0.
2	Batch 22 made a 10.3 GB logits allocation that was OOM-thrashing even under LoRA (nine recovered allocation failures in a 4-step run).	Micro-batching with gradient accumulation. DDP gradient sync suppressed on non-final micro-batches so the maths is unchanged.
3	GradScaler rejected fp16 gradients: 'Attempting to unscale FP16 gradients'. The model was loaded in pure fp16 rather than fp32 masters with fp16 compute.	dtype forced to fp32 for full+fp16; ultimately moot once run 1 moved to bf16.
4	Collator dropped mm_token_type_ids — a hardcoded key list. Qwen3-VL's forward needs it and the exact set moves between transformers versions.	Forward the entire BatchFeature rather than a fixed list of keys.
5	acc_loss was zeroed before being logged, so train/loss would have read 0.	Capture step_loss before resetting the accumulator.

Two environment issues also surfaced: the HuggingFace download needed hf_xet disabled, and the login node refused a DataLoader thread outright at load average 44 with 21 users.

6. Open decisions

Nothing below is blocking; each is a judgement call worth making deliberately.

Decision	Detail	Recommendation
Paired/unpaired coverage	With a Bernoulli(0.5) draw and 3 epochs, ~5,209 images (25%) never see one of the two conditions. Deterministic alternation on (image_index + epoch) gives 100% coverage at ZERO extra cost. Duplicating every image to see both conditions each epoch would take 2,841 steps instead of 1,419.	Alternation. Same cost, full coverage.
Number of epochs	Validation loss was already 0.0089 at step 500 (~1 epoch). 10 epochs would be 4,730 steps and ~26 h for all three, with real overfitting risk on a closed-vocabulary JSON task — and memorization degrades hardest on OOD evaluation.	Keep 3, watch val/loss, extend from the checkpoint if it is still falling.

Stale JSONL	jsonl/*.jsonl still hold the old 6-key targets. Training does not read them, but anything else that does will be silently wrong.	Regenerate from v2.
Login node	GPUs are shared: free VRAM was observed swinging from 472 MB to 70 GB within two minutes, and a thread allocation was refused at load average 44. The harness survives this by resuming, but each crash discards up to 250 steps.	sbatch on compute nodes if the node gets crowded.
Grade masking	243 gradeable-but-ungraded rows (GastroVision esophagitis 107, barretts 95; HyperKvasir barretts 41) contradict graded rows of the same finding, distinguishable only by provenance the model cannot see.	Mask the grade loss on those rows.
Duplicates	10 LIMUC contradictory pairs (same md5, conflicting labels) and 1 GastroVision image straddling train/val.	Drop before the final run.

Files

Path	Purpose
configs/target_schema_v2.json	Formal JSON Schema; all 29,938 targets validate, 0 failures
configs/train_v2.yaml	The three runs and all hyperparameters
reports/schema_key_presence.csv	29,938 x 25 — per-image key presence and values
src/endoedge/prompts_v2.py	25 role prompts, paired/unpaired construction
src/endoedge/dataset_v2.py	CSV-driven dataset and PairedCollator
scripts/train_v2.py	Trainer: full / LoRA / QLoRA
scripts/harness.py	Chaining, state, auto-resume
scripts/13_key_presence_csv.py	Regenerates the CSV
README_training.md	Launch instructions and risks